

17 – Python Classes

CLASES Y PROGRAMACIÓN ORIENTADA A OBJETOS

Las clases pueden usarse de muchas maneras diferentes. Ahora enfatizamos su uso en el contexto de la **programación orientada a objetos**. La clave de la programación orientada a objetos es pensar en *los objetos como colecciones de tanto datos como los métodos que operan sobre esos datos*.

❗ ¿Qué es un objeto?

Un **objeto** en Python es una instancia de un tipo. Cada objeto tiene dos características fundamentales:

1. **Una representación interna**, que son los **atributos de datos** (por ejemplo, `x = 3`).
2. **Una interfaz**, que está compuesta por los **métodos** que operan sobre ese objeto.

Todo en Python es un objeto, desde `123` (int) hasta `"Hola"` (str), listas, diccionarios y los objetos personalizados que definimos nosotros mismos con clases.

Las ideas que subyacen a la programación orientada a objetos tienen alrededor de 50 años, y han sido ampliamente aceptadas y practicadas durante los últimos 30 años más o menos. A mediados de los años 1970, las personas comenzaron a escribir artículos explicando los beneficios de este enfoque a la programación. Aproximadamente al mismo tiempo, los lenguajes de programación **SmallTalk** (en Xerox PARC) y **CLU** (en MIT) proporcionaron soporte lingüístico para las ideas. Pero no fue hasta la llegada de **C++** y **Java** que la programación orientada a objetos realmente despegó en la práctica.

En secciones anteriores dijimos *"Los objetos son las cosas principales que los programas de Python manipulan. Cada objeto tiene un tipo que define los tipos de cosas que los programas pueden hacer con ese objeto."* Hemos confiado en tipos incorporados como `int` y `str` y los métodos asociados con esos tipos. Pero así como los diseñadores de un lenguaje de programación pueden incorporar solo una pequeña fracción

de las funciones útiles, pueden incorporar solo una pequeña fracción de los tipos útiles. Ya hemos visto un mecanismo que permite a los programadores definir nuevas funciones; ahora veremos un mecanismo que permite a los programadores definir nuevos tipos.

Tipos de Datos Abstractos y Clases

La noción de un tipo de datos abstracto es bastante simple. Un **tipo de datos abstracto** es un *conjunto de objetos y las operaciones sobre esos objetos*. Estos están unidos de tal manera que los programadores pueden pasar un objeto de una parte de un programa a otra, y al hacerlo proporcionar acceso no solo a los atributos de datos del objeto sino también a las operaciones que facilitan la manipulación de esos datos.

Las especificaciones de esas operaciones definen una **interfaz** entre el tipo de datos abstracto y el resto del programa. La interfaz define el comportamiento de las operaciones y qué hacen, pero no cómo lo hacen. La interfaz así proporciona una **barrera de abstracción** que aísla el resto del programa de las estructuras de datos, algoritmos y código involucrados en proporcionar una realización de la abstracción del tipo.

Definición de interfaz

Un conjunto de operaciones o métodos que un tipo de dato (por ejemplo, una lista, una pila, un mapa) promete ofrecer al resto del programa, sin revelar cómo están implementadas internamente esas operaciones.

Imaginá que estás usando una **pila (stack)**. La interfaz de la pila te dice:

- **push(elemento)**: agrega un elemento a la pila.
- **pop()**: saca el último elemento agregado.
- **is_empty()**: dice si la pila está vacía.

Pero **no te dice si la pila está implementada con una lista, con un array, con punteros, ni cómo funciona internamente.**

La programación se trata de manejar la complejidad de una manera que facilite el cambio. Dos mecanismos poderosos están disponibles para lograr esto: **descomposición** y **abstracción**.

La **descomposición crea estructura en un programa**, y la **abstracción suprime el detalle**. La clave es suprimir los detalles apropiados. Aquí es donde la abstracción de datos da en el blanco. Podemos crear tipos específicos de dominio que proporcionen una abstracción conveniente. Idealmente, estos tipos capturan conceptos que serán relevantes durante la vida útil de un programa. Si comenzamos el proceso de programación ideando tipos que serán relevantes meses e incluso décadas después, tenemos una gran ventaja en mantener ese software.

Hemos estado usando tipos de datos abstractos (sin llamarlos así). Hemos escrito programas usando enteros, listas, flotantes, cadenas y diccionarios sin pensar en cómo estos tipos podrían estar implementados. Parafraseando *Le Bourgeois Gentilhomme* de Molière, "*Por ma foi, il y a plus de cent pages que nous avons utilisé ADTs, sans que nous le sachions*" ("¡Por mi fe, llevamos más de cien páginas usando ADTs sin saberlo!").

En Python, implementamos abstracciones de datos usando clases. Cada definición de clase comienza con la palabra reservada `class` seguida por el nombre de la clase y alguna información sobre cómo se relaciona con otras clases.

Considera la siguiente **definición de clase** pequeña (y totalmente inútil).

```
1 class Toy(object):
2     def __init__(self):
3         self._elems = []
4     def add(self, new_elems):
5         """new_elems is a list"""
6         self._elems += new_elems
7     def size(self):
8         return len(self._elems)
```

La primera línea indica que `Toy` es una subclase de `object`. Por ahora, ignora lo que significa ser una subclase. Volveremos a eso en breve.

Una definición de clase crea un objeto de tipo `type` y asocia con ese objeto de clase un conjunto de objetos llamados **atributos**. En este ejemplo, los tres atributos asociados con la clase son `__init__`, `add`, y `size`. Cada uno es de tipo `function`. Consecuentemente, el código,

```
1 print(type(Toy))
2 print(type(Toy.__init__), type(Toy.add), type(Toy.size))
```

Imprime

```
1 <class 'type'>
2 <class 'function'> <class 'function'> <class 'function'>
```

Como veremos, Python tiene un número de nombres de función especiales que comienzan y terminan con dos guiones bajos. Estos a menudo se denominan **métodos mágicos**. El primero de estos que examinaremos es `__init__`. Cada vez que se instancia una clase, se hace una llamada a la función `__init__` definida en esa clase. Cuando se ejecuta la línea de código

```
1 s = Toy()
```

el intérprete creará una nueva **instancia** de tipo `Toy`, y luego llamará a `Toy.__init__` con el objeto recién creado como el parámetro actual que se vincula al parámetro formal `self`. Cuando se invoca, `Toy.__init__` crea el objeto lista `_elems`, que se convierte en parte de la instancia recién creada de tipo `Toy`. (La lista se crea usando la notación ya familiar `[]`, que es simplemente una abreviación de `list()`.) La lista `_elems` se llama un **atributo de datos** de la instancia de `Toy`. El código

```
1 t1 = Toy()
2 print(type(t1))
3 print(type(t1.add))
4 t2 = Toy()
5 print(t1 is t2) #test for object identity
```

Imprime

```
1 <class '__main__.Toy'>
2 <class 'method'>
3 False
```

Nota que `t1.add` es de tipo `method`, mientras que `Toy.add` es de tipo `function`. Porque `t1.add` es un método, podemos invocarlo (y `t1.size`) usando notación de punto.

≡ Ejemplos del mundo real como objetos

Para comprender mejor los objetos y su abstracción, pensemos en ejemplos del mundo cotidiano:

- **Elevador:**

- Atributos → `current_floor`, `max_capacity`
- Métodos → `move_to(floor)`, `add_person()`, `remove_person()`

- **Empleado:**
 - Atributos → `name` , `birth_date` , `salary`
 - Métodos → `update_salary()` , `get_age()`
- **Pila de panqueques (Stack):**
 - Atributos → Lista interna de panqueques
 - Métodos → `push(pancake)` , `pop()`
- **Cola de supermercado (Queue):**
 - Atributos → Lista de clientes
 - Métodos → `enqueue(cliente)` , `dequeue()`

Una clase no debe confundirse con instancias de esa clase, así como un objeto de tipo `list` no debe confundirse con el tipo `list`. Los atributos pueden asociarse ya sea con una clase en sí misma o con instancias de una clase:

- Los atributos de clase se definen en una definición de clase; por ejemplo, `Toy.size` es un atributo de la clase `Toy`. Cuando se instancia una clase, por ejemplo, con la declaración `t = Toy()`, se crean atributos de instancia, por ejemplo, `t.size`.
- Mientras que `t.size` inicialmente está vinculado a la función `size` definida en la clase `Toy`, esa vinculación puede cambiarse durante el curso de una computación. Por ejemplo, podrías (pero definitivamente ¡no deberías!) cambiar la vinculación ejecutando `t.size = 3`.
- Cuando los atributos de datos se asocian con una clase, los llamamos **variables de clase**. Cuando se asocian con una instancia, los llamamos **variables de instancia**. Por ejemplo, `_elems` es una variable de instancia porque para cada instancia de la clase `Toy`, `_elems` está vinculado a una lista diferente. Hasta ahora, no hemos visto una variable de clase.

Ahora, considera el código

```
1  t1 = Toy()
2  t2 = Toy()
3  t1.add([3, 4])
4  t2.add([4])
5  print(t1.size() + t2.size())
```

Dado que cada instancia de `Toy` es un objeto diferente, cada instancia de tipo `Toy` tendrá un atributo `_elems` diferente. Por lo tanto, el código imprime `3`.

A primera vista, algo parece ser inconsistente en este código. Parece como si cada método estuviera siendo llamado con un argumento. Por ejemplo, `add` tiene dos parámetros formales, pero parecemos estar llamándolo con solo un parámetro actual. Esto es un artefacto de usar notación de punto para invocar un método asociado con una instancia o una clase. El objeto asociado con la expresión que precede al punto se pasa implícitamente como el primer parámetro al método. Seguimos la convención de usar `self` como el nombre del parámetro formal al cual este parámetro actual está vinculado. Los programadores de Python observan esta convención casi universalmente, y recomendamos encarecidamente que la uses también.

Otra convención común es comenzar el nombre de los atributos de datos con un guion bajo. *Usamos el guion bajo inicial para indicar que el atributo es privado a la clase*, es decir, no debería ser accedido directamente fuera de la clase.

Ahora, veamos un ejemplo más interesante. El siguiente código contiene una **definición de clase** que proporciona una implementación directa de una abstracción conjunto-de-enteros llamada `IntSet`. (Dado que Python tiene un tipo incorporado `set`, esta implementación es tanto innecesaria como innecesariamente complicada. Sin embargo, es pedagógicamente útil.)

Clase IntSet

```
1  class IntSet(object):
2      """An IntSet is a set of integers"""
3      #Information about the implementation (not the abstraction):
4      #Value of a set is represented by a list of ints, self._vals.
5      #Each int in a set occurs in self._vals exactly once.
6
7      def __init__(self):
8          """Create an empty set of integers"""
9          self._vals = []
10
11     def insert(self, e):
12         """Assumes e is an integer and inserts e into self"""
13         if e not in self._vals:
14             self._vals.append(e)
15
16     def member(self, e):
17         """Assumes e is an integer
18         Returns True if e is in self, and False otherwise"""
19         return e in self._vals
20
21     def remove(self, e):
22         """Assumes e is an integer and removes e from self
23         Raises ValueError if e is not in self"""
24         try:
```

```

25         self._vals.remove(e)
26     except:
27         raise ValueError(str(e) + ' not found')
28
29     def get_members(self):
30         """Returns a list containing the elements of self.
31         Nothing can be assumed about the order of the elements"""
32         return self._vals[:]
33
34     def __str__(self):
35         """Returns a string representation of self"""
36         if self._vals == []:
37             return '{}'
38         self._vals.sort()
39         result = ''
40         for e in self._vals:
41             result = result + str(e) + ', '
42         return '{' + result[:-1] + '}'

```

Nota que la cadena de documentación (el comentario encerrado en `"""`) en la parte superior de la definición de clase describe la abstracción proporcionada por la clase, no información sobre cómo la clase está implementada. En contraste, los comentarios debajo de la cadena de documentación contienen información sobre la implementación. Esa información está dirigida a programadores que podrían querer modificar la implementación o construir subclases de la clase, no a programadores que simplemente quieran usar la abstracción.

Como hemos visto, los métodos asociados con una instancia de una clase pueden invocarse usando notación de punto. Por ejemplo, el código

```

1  s = IntSet()
2  s.insert(3)
3  print(s.member(3))

```

crea una nueva instancia de `IntSet`, inserta el entero 3 en ese `IntSet`, y luego imprime `True`.

La *abstracción de datos logra independencia de representación*. Piensa en la implementación de un tipo abstracto como teniendo varios componentes:

- Implementaciones de los métodos del tipo.
- Estructuras de datos que juntas codifican valores del tipo.
- Convenciones sobre cómo las implementaciones de los métodos van a usar las estructuras de datos; una convención clave es capturada por el invariante de representación.

El **invariante de representación** define qué valores de los atributos de datos corresponden a representaciones válidas de instancias de clase. El invariante de representación para `IntSet` es que `_vals` no contiene duplicados. La implementación de `__init__` es responsable de establecer el invariante (que se cumple para la lista vacía), y los otros métodos son responsables de mantener ese invariante. Por eso `insert` agrega `e` solo si no está ya en `self._vals`.

La implementación de `remove` explota la suposición de que el invariante de representación se satisface cuando se invoca `remove`. Llama a `list.remove` solo una vez, ya que el invariante de representación garantiza que hay como máximo una ocurrencia de `e` en `self._vals`.

El último método definido en la clase, `__str__`, es otro de esos métodos especiales `__`. El método `__str__` de una clase se invoca cuando un programa convierte una instancia de esa clase a una cadena llamando `str`. Por lo tanto, cuando se ejecuta el comando `print`, la función `__str__` asociada con el objeto a ser impreso es invocada. Por ejemplo, el código

```
1 s = Int_set()
2 s.insert(3)
3 s.insert(4)
4 print(str(s))
5 print('El valor de s es', s)
```

imprimirá

```
1 {3,4}
2 El valor de s es {3,4}
```

(Si no se hubiera definido el método `__str__`, ejecutar `print(s)` habría causado que se imprimiera algo como `<__main__.Int_set object at 0x1663510> .`)

🔗 Visualización de instancias en memoria

Cuando creamos una instancia de una clase, como esta:

```
1 class Coordinate:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6 c = Coordinate(3, 4)
```


Internamente, Python reserva memoria para un nuevo objeto que contiene sus propios valores de `x` y `y`.

```
1  c  →  Objeto Coordinate
2         └─ x: 3
3         └─ y: 4
```

Cuando hacemos `print(c.x)`, Python busca en la estructura de `c` el valor asociado a `x`.

Cada instancia creada con `Coordinate(...)` tiene su propia copia de `x` y `y`.

También podemos ver que:

```
1  origin = Coordinate(0, 0)
2  print(c.x) # 3
3  print(origin.x) # 0
```

Esto demuestra que cada instancia es independiente, aunque usen la misma clase como "molde".

Ejercicio manual

Agrega un método que satisfaga la especificación a continuación a la clase `Int_set`.

```
1  def union(self, other):
2      """other es un Int_set
3          muta self para que contenga exactamente los
4          elementos en self
5          más los elementos en other."""
6      for e in other.get_members():
7          if e not in self._vals:
8              self._vals.append(e)
```

Métodos Mágicos y Tipos con Hash

Uno de los objetivos de diseño de Python era permitir a los programadores usar clases para definir nuevos tipos que fueran tan fáciles de usar como los tipos incorporados de Python. Usar métodos mágicos para proporcionar definiciones específicas de clase de funciones incorporadas como `str` y `len` juega un papel importante en lograr este objetivo.

Los métodos mágicos también pueden ser usados para proporcionar definiciones específicas de clase para operadores infijos como `==` y `+`. Los nombres de los métodos disponibles para operadores infijos son:

1	<code>+: __add__</code>	<code>*: __mul__</code>	<code>/: __truediv__</code>
2	<code>-: __sub__</code>	<code>//: __floordiv__</code>	<code>?: __mod__</code>
3	<code>**: __pow__</code>	<code> : __or__</code>	<code><: __lt__</code>
4	<code><<: __lshift__</code>	<code>^: __xor__</code>	<code>>: __gt__</code>
5	<code>>>: __rshift__</code>	<code>==: __eq__</code>	<code><=: __le__</code>
6	<code>&: __and__</code>	<code>!=: __ne__</code>	<code>>=: __ge__</code>

Puedes asociar cualquier implementación que desees con estos operadores. Si quisieras, podrías implementar `+` como sustracción, `<` como exponenciación, etc. Sin embargo, recomendamos que resistas la oportunidad de ser imaginativo, y te apegues a implementaciones consistentes con los significados convencionales de estos operadores.

Usando métodos mágicos

```
1 class Toy(object):
2
3     def __init__(self):
4         self._elems = []
5
6     def add(self, new_elems):
7         """new_elems es una lista"""
8         self._elems += new_elems
9
10    def __len__(self):
11        return len(self._elems)
12
13    def __add__(self, other):
14        new_toy = Toy()
15        new_toy._elems = self._elems + other._elems
16        return new_toy
17
18    def __eq__(self, other):
19        return self._elems == other._elems
20
21    def __str__(self):
22        return str(self._elems)
23
24    def __hash__(self):
25        return id(self)
26
27    t1 = Toy()
28    t2 = Toy()
29    t1.add([1, 2])
30    t2.add([3, 4])
31    t3 = t1 + t2
32
33    print('El valor de t3 es', t3)
```

```

34 print('La longitud de t3 es', len(t3))
35
36 d = {t1: 'A', t2: 'B'}
37
38 print('El valor', d[t1], 'está asociado con la clave t1 en d.')

```

Cuando se ejecuta el código imprime

```

1 El valor de t3 es [1, 2, 3, 4]
2 La longitud de t3 es 4
3 El valor A está asociado con la clave t1 en d.

```

Podemos usar instancias de `Toy` como claves de diccionario porque definimos una función `__hash__` para la clase. Si hubiéramos definido una función `__eq__` y no hubiéramos definido una función `__hash__`, el código habría generado el mensaje de error `unhashable type: 'Toy'` cuando intentamos crear un diccionario usando `t1` y `t2` como claves. Cuando se proporciona un `__hash__` definido por el usuario, debe asegurar que el valor hash de un objeto sea constante durante toda la vida de ese objeto.

Todas las instancias de clases definidas por el usuario que no definen explícitamente `__eq__` usan identidad de objeto para `==` y son hashables. Si no se proporciona un método `__hash__`, el valor hash del objeto se deriva de la identidad del objeto.

Ejercicio manual

Reemplaza el método `union` que agregaste a `Int_set` por un método que permita a los clientes de `Int_set` usar el operador `+` para denotar unión de conjuntos.

```

1 def __add__(self, other):
2     result = IntSet()
3
4     for e in self.get_members():
5         result.insert(e)
6     for e in other.get_members():
7         result.insert(e)
8
9     return result

```

Diseñando Programas Usando Tipos de Datos Abstractos

Los tipos de datos abstractos son muy importantes. Llevan a una forma diferente de pensar sobre la organización de programas grandes. Cuando pensamos sobre el mundo, no pensamos en abstracciones. En el mundo de las finanzas, la gente habla sobre acciones y bonos. En el mundo de la biología, la gente habla sobre proteínas y residuos. Cuando tratamos de entender conceptos como estos, mentalmente reunimos algunos de los datos relevantes y características de estos tipos de objetos en un paquete intelectual. Por ejemplo, pensamos en los bonos como teniendo una tasa de interés, una fecha de vencimiento y un precio como atributos de datos. También pensamos en los bonos como teniendo operaciones tales como "establecer precio" y "calcular rendimiento al vencimiento." Los tipos de datos abstractos nos permiten incorporar este tipo de organización en el diseño de programas.

La abstracción de datos *alienta a los diseñadores de programas a enfocarse en la centralidad de los objetos de datos en lugar de las funciones*. Pensar sobre un programa más como una colección de tipos que como una colección de funciones lleva a un principio organizativo profundamente diferente. Entre otras cosas, nos alienta a pensar sobre la programación como un proceso de combinar fragmentos relativamente grandes, ya que los datos abstractos típicamente abarcan más funcionalidad que las funciones individuales. Esto, a su vez, *nos lleva a pensar en la esencia de la programación como un proceso no de escribir líneas individuales de código, sino de componer abstracciones*.

La disponibilidad de abstracciones reutilizables no solo reduce el tiempo de desarrollo, sino que usualmente lleva a programas más confiables, porque el software maduro es usualmente más confiable que el software nuevo. Por muchos años, las únicas librerías de programas de uso común eran estadísticas o científicas. Hoy, sin embargo, hay una gran gama de librerías de programas disponibles (especialmente para Python), a menudo basadas en un rico conjunto de abstracciones de datos.

Usando Clases para Llevar un Registro de Estudiantes y Profesores

Como ejemplo del uso de clases, imagina que estás diseñando un programa para ayudar a llevar un registro de todos los estudiantes y profesores en una universidad. Ciertamente es posible escribir tal programa sin usar abstracción de datos. Cada estudiante tendría un nombre familiar, un nombre dado, una dirección de casa, un año,

algunas calificaciones, etc. Todos estos datos podrían mantenerse en una combinación de listas y diccionarios. Llevar registro del profesorado y personal requeriría algunas estructuras de datos similares y algunas estructuras de datos diferentes, por ej., estructuras de datos para llevar registro del historial de salarios de las cosas!

Antes de apresurarse a diseñar un montón de estructuras de datos, pensemos sobre algunas abstracciones que podrían resultar útiles. ¿Hay una abstracción que cubra los atributos comunes de estudiantes, profesores y personal? Algunos argumentarían que todos son humanos. El siguiente código contiene una clase que incorpora dos atributos comunes (nombre y fecha de nacimiento) de los humanos. Usa el módulo estándar de Python `datetime`, que proporciona muchos métodos convenientes para crear y manipular fechas.

Clase Person

```
1  class Person(object):
2      def __init__(self, name):
3          """Asume que name es una cadena. Crea una persona"""
4          self._name = name
5          try:
6              last_blank = name.rindex(' ')
7              self._last_name = name[last_blank+1:]
8          except:
9              self._last_name = name
10         self._birthday = None
11
12     def get_name(self):
13         """Devuelve el nombre completo de self"""
14         return self._name
15
16     def get_last_name(self):
17         """Devuelve el apellido de self"""
18         return self._last_name
19
20     def set_birthday(self, birthdate):
21         """Asume que birthdate es de tipo datetime.date
22            Establece el cumpleaños de self a birthdate"""
23         self._birthday = birthdate
24
25     def get_age(self):
26         """Devuelve la edad actual de self en días"""
27         if self._birthday == None:
28             raise ValueError
29         return (datetime.date.today() - self._birthday).days
30
31     def __lt__(self, other):
32         """Asume que other es una Person
33            Devuelve True si self precede a other en orden alfabético,
34            y False en caso contrario.
```

```

35         La comparación se basa en los apellidos,
36         pero si estos son los mismos,
37         los nombres completos son comparados."""
38         if self._last_name == other._last_name:
39             return self._name < other._name
40         return self._last_name < other._last_name
41
42     def __str__(self):
43         """Devuelve el nombre de self"""
44         return self._name

```

El siguiente código usa `Person` y `datetime`.

```

1  me = Person('Michael Gutttag')
2  him = Person('Barack Hussein Obama')
3  her = Person('Madonna')
4
5  print(him.get_last_name())
6
7  him.set_birthday(datetime.date(1961, 8, 4))
8  her.set_birthday(datetime.date(1958, 8, 16))
9
10 print(him.get_name(), 'is', him.get_age(), 'days old')

```

Nota que siempre que se instancia `Person`, se suministra un argumento a la función `__init__`. En general, cuando instanciamos una clase necesitamos ver la especificación de la función `__init__` para esa clase para saber qué argumentos suministrar y qué propiedades deberían tener esos argumentos.

Ejecutar el código anterior crea tres instancias de la clase `Person`. Podemos acceder a información sobre estas instancias usando los métodos asociados con ellas. Por ejemplo, `him.get_last_name()` devuelve `'Obama'`. La expresión `him.last_name` también devolverá `'Obama'`; sin embargo, escribir expresiones que accedan directamente a variables de instancia se considera una mala práctica, y debería evitarse. De manera similar, no hay una forma apropiada para que un usuario de la abstracción `Person` extraiga el cumpleaños de una persona, a pesar del hecho de que la implementación contiene un atributo con ese valor. (Por supuesto, sería fácil agregar un método `get_birthday` a la clase.) Hay, sin embargo, una forma de extraer información que depende del cumpleaños de la persona, como se ilustra en la última declaración `print` en el código anterior.

La clase `Person` proporciona una definición específica de `Person` para otro método especialmente nombrado, `__lt__`. Este método sobrecarga el operador `<`. El método `Person.__lt__` es llamado cuando el primer argumento del operador `<` es de tipo `Person`. El método `__lt__` en la

clase `Person` está implementado usando el operador binario `<` de tipo `str`. La expresión `self._name.__lt__(other._name)` es una forma abreviada para `self._name < other._name`. Como `self._name` es de tipo `str`, este método `__lt__` es el asociado con el tipo `str`.

Además de proporcionar la conveniencia sintáctica de escribir expresiones infijas que usan `<`, esta sobrecarga proporciona acceso automático a cualquier método polimórfico definido usando `__lt__`. El método incorporado `sort` es uno de esos métodos. Entonces, por ejemplo, si `pList` esta compuesto por elementos de tipo `Person`, la llamada `p_list.sort()` ordenará esa lista usando el método `__lt__` definido en la clase `Person`. Por lo tanto, el código

```
1  pList = [me, him, her]
2  for p in pList:
3      print(p)
4  pList.sort()
5  for p in pList:
6      print(p)
```

imprimirá

```
1  Michael Gutttag
2  Barack Hussein Obama
3  Madonna
4  Michael Gutttag
5  Madonna
6  Barack Hussein Obama
```

Ejercicio manual de la lectura

Implemente la clase que cumpla las siguientes especificaciones:

```
1  class Circle():
2      def __init__(self, radius):
3          """ Initializes self with radius """
4          self._radius = radius
5
6      def get_radius(self):
7          """ Returns the radius of self """
8          return self._radius
9
10     def set_radius(self, radius):
11         """ radius is a number
12         Changes the radius of self to radius """
13         self._radius = radius
14
```

```
15     def get_area(self):
16         """ Returns the area of self using pi = 3.14 """
17         return 3.14 * self._radius ** 2
18
19     def equal(self, c):
20         """ c is a Circle object
21         Returns True if self and c have the same radius value """
22         return self._radius == c.get_radius()
23
24     def bigger(self, c):
25         """ c is a Circle object
26         Returns self or c, the Circle object with the bigger radius """
27         if self._radius > c.get_radius():
28             return self
29         else:
30             return c
```